



Cairo University

Faculty of Engineering

Computer Engineering Department

Pattern Recognition and Neural Networks

Project Document: Robust Human Activity Recognition (HAR) System



Spring 2026

Contents

1. **Objectives**
2. **Project: Robust Human Activity Recognition System**
    - 2.1 System Specs
    - 2.2 Dataset
3. **Team Formation**
4. **Research Team Proposal**
5. **Final Deliverables**
6. **Rules and Instructions**
7. **Grading Criteria**
8. **FAQ**
9. **Delivery Instructions**
    - 9.1 Input Format
    - 9.2 Output Format

1. Objectives

By the end of this project, you should be able to:

1. Analyze complex multi-attribute tabular telemetry data, extract meaningful patterns, and choose the most appropriate preprocessing and feature normalization methods.
2. Assess and compare the performance of diverse machine learning and pattern recognition techniques under real-world domain shifts.
3. Design, implement, and optimize a robust end-to-end machine learning pipeline capable of generalizing to unseen hardware environments.

2. Project: Robust Human Activity Recognition System

In this project, you are required to implement a **Robust Human Activity Recognition (HAR) System** using smartphone inertial sensor telemetry. Given tabular records extracted from smartphone accelerometers and gyroscopes, your system is required to classify each instance into one of **6 discrete human activity classes** (from 0 to 5).

The dataset captures multi-axial movement dynamics and postural states. Each record consists of **562 continuous numerical features** (combining time-domain statistical aggregates and frequency-domain spectral transformations) mapped to one of the following activity labels:

Label	Activity Class	Description
0	Walking	Dynamic forward locomotion with cyclic limb oscillations
1	Walking Upstairs	High-energy vertical ascent with distinct gravitational load shifts
2	Walking Downstairs	Deceleration-heavy descent exhibiting high impact shock profiles
3	Sitting	Stationary postural state with minimal acceleration variance
4	Standing	Upright static posture with subtle postural sway
5	Laying	Horizontal resting posture with stable orientation alignments

2.1 System Specs

Your system must handle real-world variations and multi-device sensor heterogeneity as robustly as possible. You are required to implement a complete, principled machine learning pipeline, including (but not limited to):

- Preprocessing and feature normalization module.
- Feature selection / dimensionality reduction module.
- Model selection, hyperparameter tuning, and cross-validation training module.
- Performance analysis and error evaluation module.

Your preprocessing module should account for real-world sensor challenges such as:

- Hardware calibration offsets and sensor sensitivity variations across different smartphone manufacturers.
- Non-stationary noise variance and feature drift.
- Outlier detection and robust scaling.

You will need to research the topic, review relevant academic literature on domain generalization and inertial sensor telemetry, and conduct a thorough literature survey to identify optimal approaches and architectures to maximize your model's robustness and accuracy on unseen data.

2.2 Dataset

The dataset can be downloaded from this drive link: radar_project_data.7z
You are expected to structure your local development workflow by partitioning your data appropriately:

- **Training set** (Device_A and Device_B folders): Provided training records capturing multi-device telemetry distributions.
- **Validation set**: Reserved split for local cross-validation and hyperparameter tuning.
- **Test set**: Held-out unseen evaluation data (\data) used for final leaderboard ranking.

3. Team Formation

- A team should be formed of **3 to 4 members**. No team should have more than 4 members under any condition.
- Please register your team number in the provided course spreadsheets.
- All team members must attend the final project discussion. If a team member fails to show up for the project discussion, they will be regarded as a zero contributor for this project.

4. Research Team Proposal

Optionally, one or more teams may work on an innovative research idea related to the project problem. Such a team will be directly supervised by the course instructor, and their primary goal will be to publish a research paper on a novel contribution related to robust pattern recognition or domain generalization.
These teams will not participate in the standard leaderboard competition, and their grades will not be affected by leaderboard rankings. Instead, they will be graded based on their direct contributions to the research paper. Teams interested in this track must submit a formal proposal to the instructor outlining their motivation and proposed contribution. Selected proposals will receive approval to proceed.

5. Final Deliverables

By the end of this project, each team must submit the following items:

1. **A Formal PDF Report:** Following the standard course template and structure (available via Overleaf: (https://www.overleaf.com). The report must thoroughly cover:
    - **Abstract**
    - **Introduction**
    - **System Overview:** Project pipeline, models/algorithms evaluated, architectural decisions.

- **Experimental Setup:** Preprocessing steps, feature selection criteria, cross-validation strategy, hyperparameters, and evaluation metrics.
- **Results:** Findings, comparative analysis, error analysis (confusion matrix insights, failure modes).
- **Conclusion:** Summary of system capabilities, limitations, and future work directions.
- **Workload Distribution:** Detailed breakdown of contributions by each team member.

2. **Source Code (code_[team number].zip):** A zipped archive containing:
    - The complete source code.
    - A comprehensive readme.md detailing setup, dependencies, and execution instructions.
    - A requirements.txt file listing all required Python libraries.
    - An executable script or Python module named **infer.py**, which accepts the path to the evaluation data directory as an input argument and generates the required prediction files.

3. **Pre-trained Model File:** The trained model must be saved (preferably as a .pkl pickle file or standard state dictionary) and placed in the correct relative path within your code directory so it is loaded automatically upon execution.
4. **Dockerfile:** A valid Dockerfile allowing the course staff to replicate your exact software environment and dependencies seamlessly.
5. **BONUS (Optional):** Deploy your trained model on a public cloud service (Hugging Face Spaces is recommended) and provide the live link in your report and readme.md.

6. Rules and Instructions

All students must adhere strictly to the following rules:

1. All project artifacts must be submitted via the designated course Drive link.
2. Do not print any physical documents or reports. All submissions are strictly electronic.
3. Late submissions will incur the official course late penalty.
4. **Academic Integrity:** Any sign of cheating or plagiarism will result in a **ZERO** grade for the project for all involved teams, alongside an additional grade penalty of up to 50% of the total project score.
5. Workload must be distributed fairly and equitably among team members. Non-contributing members will face individual grading penalties.
6. Attendance at the final discussion session is mandatory for all team members. Absent members will receive a zero in the discussion evaluation.

7. Grading Criteria

The project grade is distributed across two primary components:

1. **Report and Discussion (30%):** Evaluates the quality of the research, depth of understanding during the oral discussion, and active participation by all team members through the phases of the project (Problem Understanding & System Design, Preprocessing & Feature Engineering, Model Selection & Training, Experimental Evaluation & Analysis, Conclusion & Discussion).
2. **Project Performance (70%):** Evaluates system performance on the hidden test set. Both the Classification Accuracy and Running Time are factored into the final leaderboard ranking using a competitive ranking formula.
   Take care that your choice of programming language and algorithmic efficiency will directly impact

8. FAQ

1. What programming language(s) can I use?

You may use any programming language of your choice, though Python (with libraries such as Scikit-Learn, XGBoost, PyTorch, NumPy, and Pandas) is strongly recommended.

2. Are there any restrictions on techniques or approaches?

You are entirely free to use any machine learning or pattern recognition approach you find appropriate. It is your task as engineers to discover the optimal combination of preprocessing, feature engineering, and classification models. You are not limited to classical methods (such as SVMs, Random Forests, KNN, or Multi-Layer Perceptrons) and are fully allowed to use deep learning models (such as 1D CNNs or MLPs), subject to a maximum limit of **2 billion parameters** running concurrently.

3. How can I find relevant research papers?

You can access extensive academic databases and journals by setting up an account on the **Egyptian Knowledge Bank (EKB)** using your university-provided student email. You can also utilize academic search engines such as Google Scholar or IEEE Xplore.

4. How will our project be tested?

Your inference pipeline will be evaluated against a held-out test set (\data) that you have not seen during training. This ensures a standardized, unbiased comparison across all competing teams.

5. How will our project be ranked?

Teams will be ranked on a competitive leaderboard based on classification accuracy and running time. Accuracy carries the dominant weight. For example, a model achieving 92.0% accuracy with a 5-second runtime will rank above a model achieving 80.0% accuracy with a 2-second runtime.

Delivery Instructions

9.1 Input Format

Your submitted inference script (infer.py) will be evaluated automatically against the hidden test directory. The evaluation engine will provide a folder named data. Inside this directory, you will find **N** individual CSV files representing test samples, named sequentially:

```
\data
├── 1.csv
├── 2.csv
├── 3.csv
├── ...
└── N.csv
```

Important: Your script must process the test CSV files in a strictly increasing numerical order (1, 2, 3, ..., **N**). Generating predictions out of order or failing to sort correctly will lead to evaluation mismatch.

9.2 Output Format

Your execution pipeline must generate **two text files** in the working directory:

1. results.txt
2. time.txt

9.2.1 Results (results.txt)

In results.txt, output the model's predicted class label for each of the N test samples, placing **exactly one prediction per line**. Each prediction must be an integer from **0 to 5** corresponding to the predicted activity class.

Example format:

```
2
3
0
0
1
5
3
4
1
```

(This indicates that sample 1 is classified as Class 2, sample 2 as Class 3, and so on).

Important Notes for results.txt:

1. Maintain strict correspondence with the input file order (1.csv corresponds to line 1, 2.csv to line 2, etc.).

2. The file must contain **only** the integer predictions—no extra headers, labels, or text.
3. Do not include test sample indices.
4. Do not output all predictions on a single line.

9.2.2 Time (time.txt)

In time.txt, output the elapsed execution time (in seconds) for each test iteration (including preprocessing and inference) as a single decimal number per line.

Example format:

0.045
0.038
0.042
0.051
0.039

(This indicates that processing the first test sample took 0.045 seconds, the second took 0.038 seconds, and so on).

Important Notes for time.txt:

1. Place your timer start immediately after loading the test CSV file and your timer stop immediately after generating the prediction. Record the difference (the pure pipeline runtime for a single iteration excluding disk I/O overhead).
2. All running times must be **rounded to three decimal places**.
3. The file must contain **only** the running times—no extra text or labels.
4. Do not include test sample numbers or combine outputs onto a single line.
5. Generate the physical files results.txt and time.txt automatically upon completion of the inference script.